

Titanic Survival Prediction Model





Table of Content



Team Members



Abstract



Introduction



Literature Survey



Problem Statement



Minimum Hardware Requirement



Minimum Software Requirement



Methodology Used



Algorithm



Design Framework



Implementation



Testing



Conclusion



Future Scope

Abstract

Collect Data

Clean & Process

Explore & Visualize

Logistic Regression

Prediction

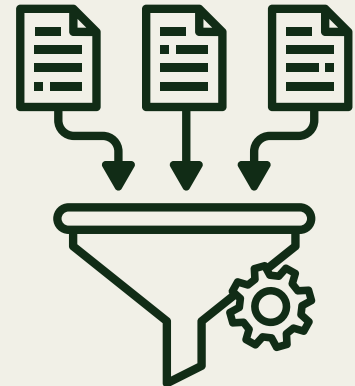
- This project predicts Titanic passenger survival using Machine Learning.
- The Titanic dataset includes features such as age, gender, passenger class, and fare.
- Data preprocessing and feature engineering were performed to improve model accuracy.
- Logistic Regression was used for binary classification.
- Model performance was evaluated using metrics like Accuracy, Precision, Recall, and F1-Score.
- A Streamlit application was developed for interactive survival prediction.

Introduction

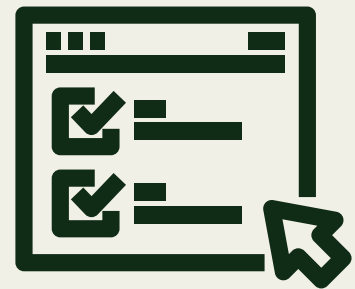
- The Titanic dataset is one of the most popular datasets used in Machine Learning.
- It contains passenger information and their survival status during the Titanic disaster.
- The project focuses on predicting survival using passenger attributes such as age, gender, and class.
- Machine Learning helps identify patterns and make accurate predictions from data.

Passenger	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	0	3	Braund, M	male	22	1	0	A/5 21171	7.25		S
2	1	1	Cumings, J	female	38	1	0	PC 17599	71.2833	C85	C
3	1	3	Heikkinen, S	female	26	0	0	STON/O2.	7.925		S
4	1	1	Futrelle, M	female	35	1	0	113803	53.1	C123	S
5	0	3	Allen, Mr.	male	35	0	0	373450	8.05		S

Literature Survey



Many researchers have worked on the Titanic dataset using different Machine Learning algorithms.



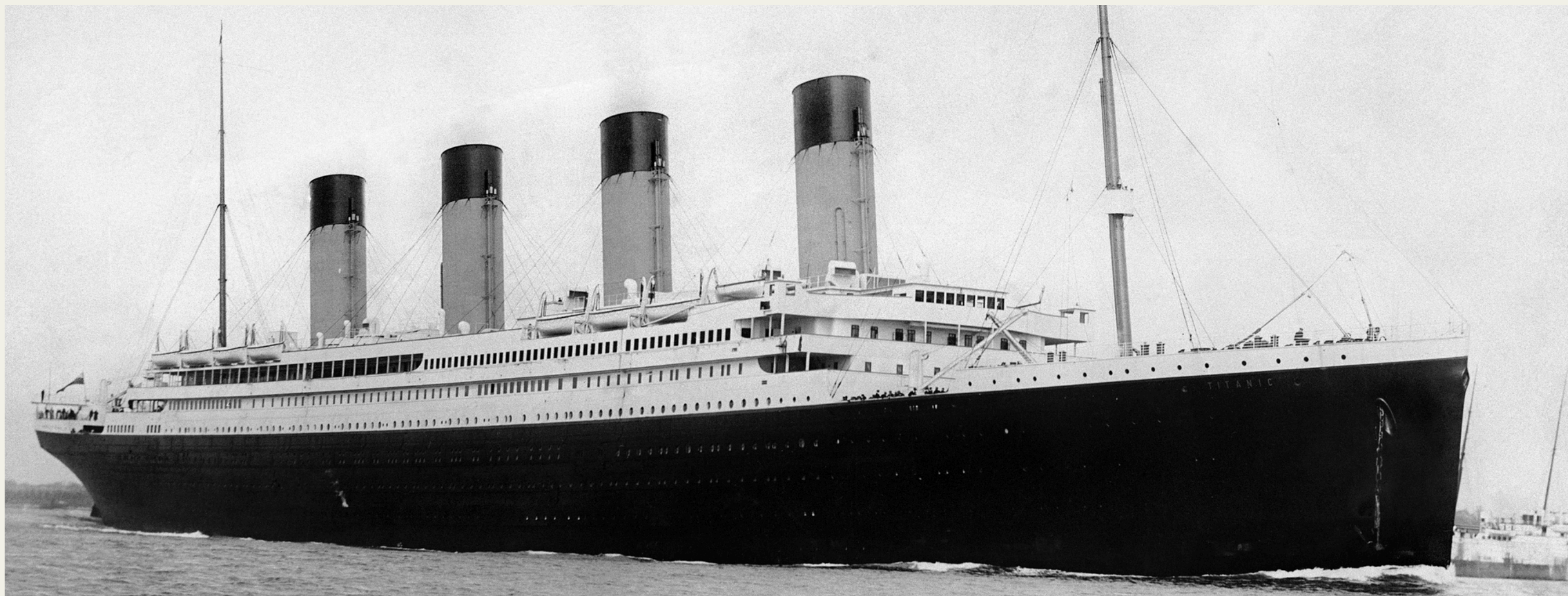
Many studies show that Logistic Regression performs effectively for binary classification, especially after proper preprocessing and feature selection



Previous studies show that feature selection and preprocessing improve model accuracy.

Problem Statement

DEVELOP A MACHINE LEARNING MODEL TO PREDICT TITANIC PASSENGER SURVIVAL BASED ON PASSENGER ATTRIBUTES USING LOGISTIC REGRESSION.



Software Requirements



Processor: Intel Core i3 (Min), i5



Graphics: Integrated graphics sufficient



RAM: 4 GB (Min), 8 GB recommended



GPU: Dedicated GPU not required for this project



Storage: 5 GB free space (Min), 10 GB+ recommended



Network: Internet needed for fastest deployment

Hardware Requirements



Operating System: Windows, macOS, or Linux



Programming Language: Python 3.6 or above



Libraries: Pandas, NumPy, Scikit-learn, Matplotlib



Development Environment: VS Code, PyCharm, Jupyter Notebook

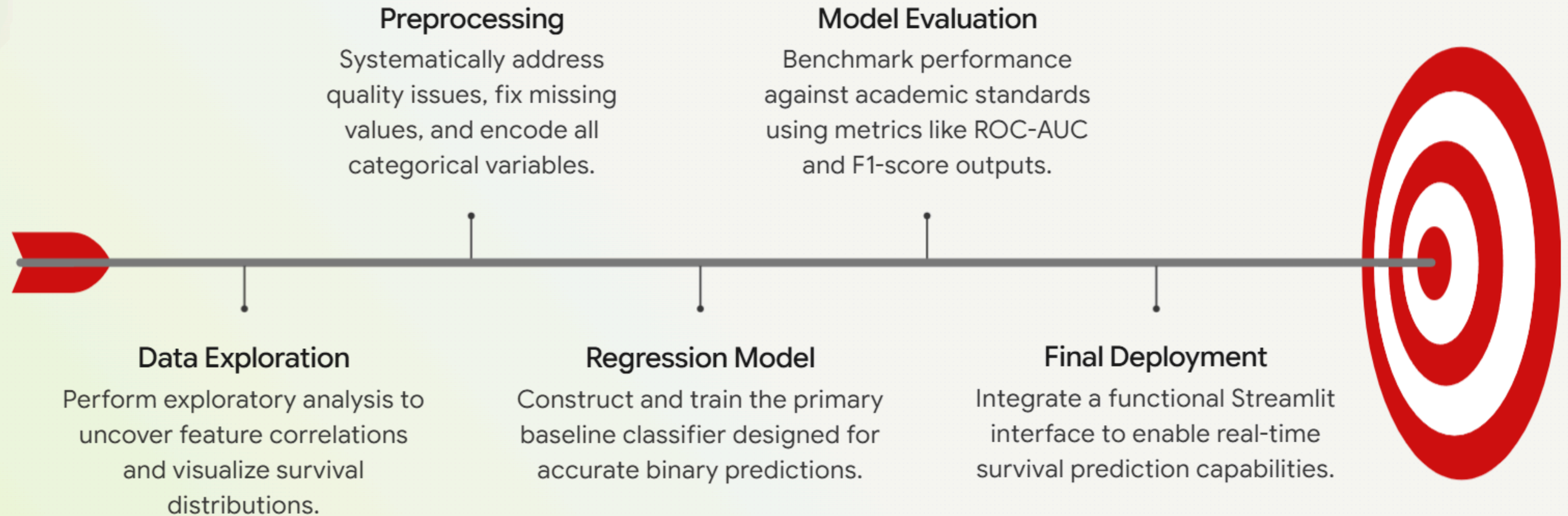


Package Manager: pip or conda



Deployment Tool: Streamlit

Methodology Used



Algorithm Used

Logistic Regression

An optimal, efficient approach for binary classification tasks, utilizing probability mapping to determine outcomes based on structured tabular data analysis.

Sigmoid Function

Maps any real-valued number into a 0 to 1 range, accurately predicting the probability of survival.

Assumptions

The model relies on a linear relationship between the independent variables and survival log-odds.



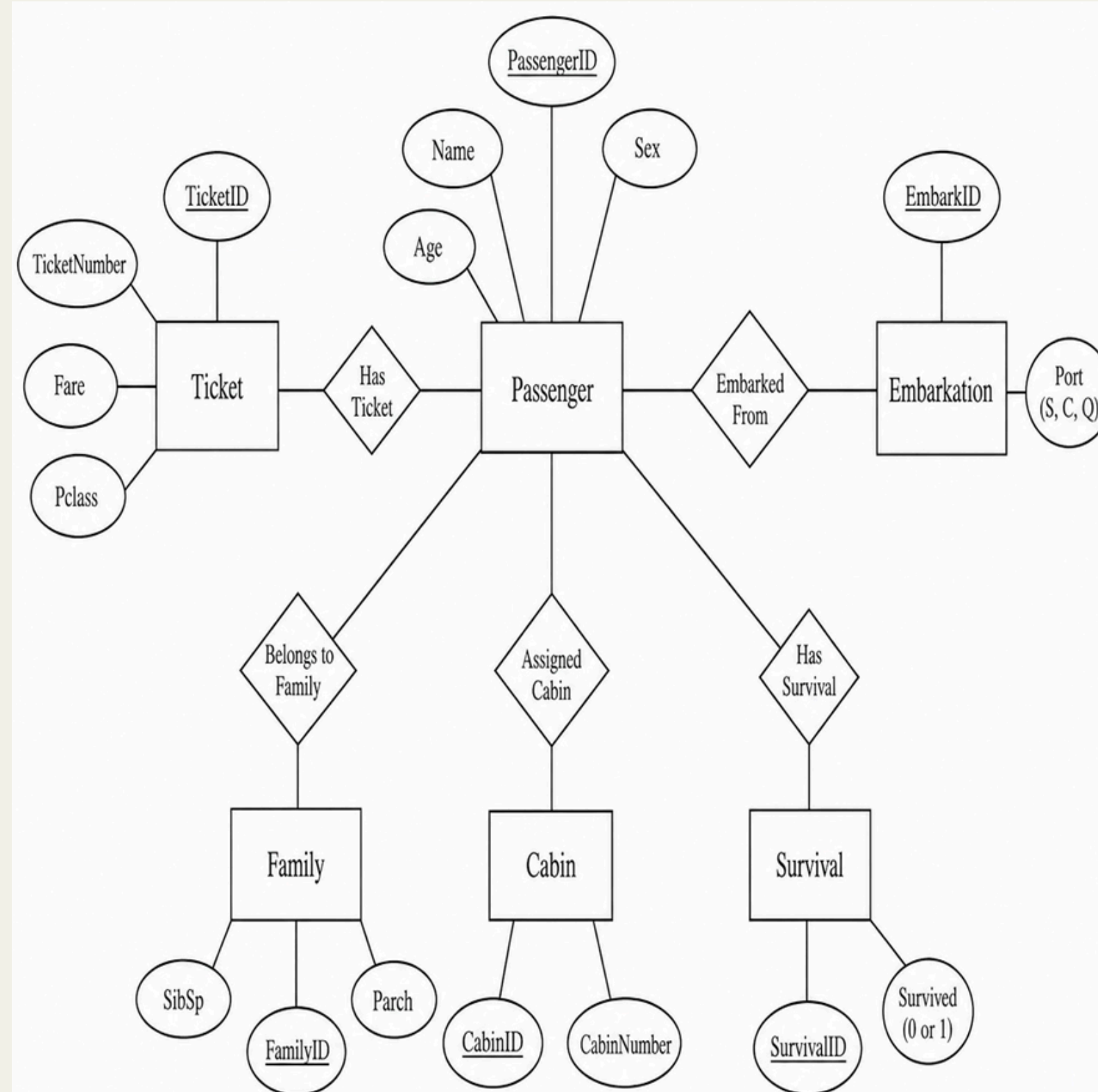
Interpretability

Model coefficients quantify the exact impact of key features like Sex or Pclass on survival odds.

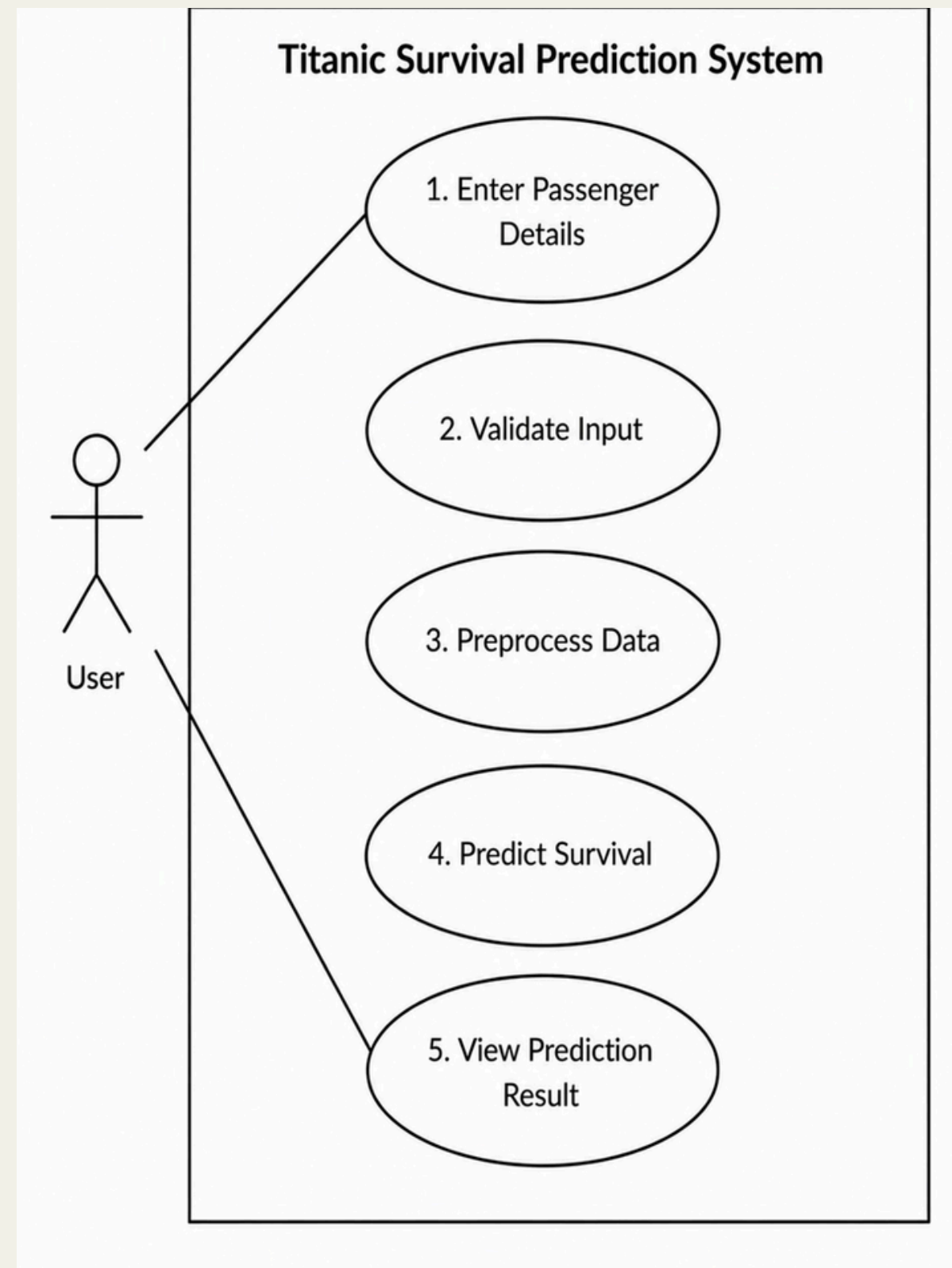
Efficiency

Requires less computational power than deep learning while delivering a strong baseline for data.

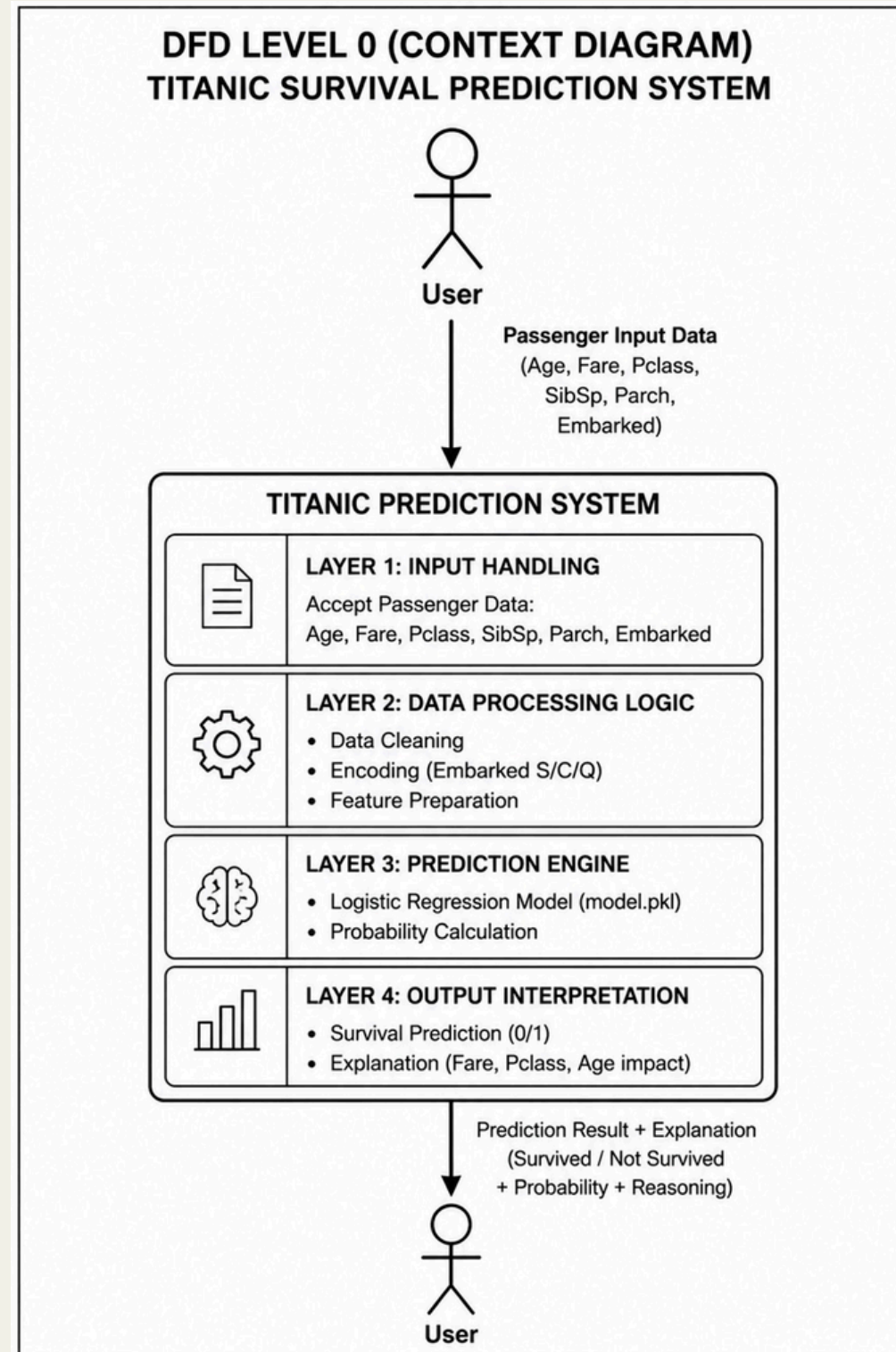
Design Framework -ER Diagram



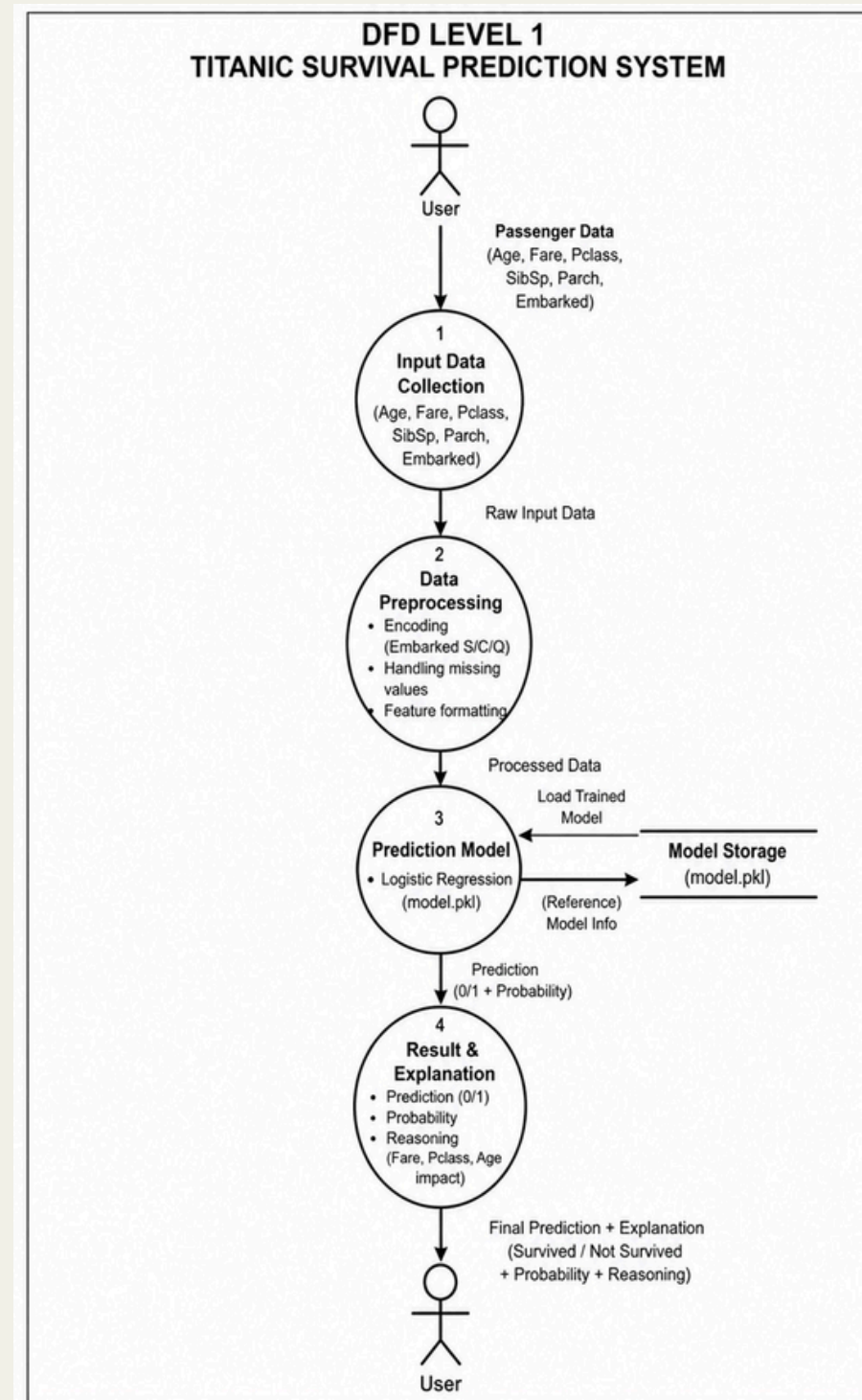
Design Framework - Use Case Diagram



Design Framework - 0 Data Flow Diagram



Design Framework - 1 Level Data Flow Diagram



Implementation

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (8,5)
```

```
In [2]: train = pd.read_csv("Titanic_train.csv")

print("Initial Shape:", train.shape)
train.head()
```

Initial Shape: (891, 12)

```
Out[2]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Implementation

```
In [11]: train['Age'] = train['Age'].fillna(train['Age'].median())

embarked_mode = train['Embarked'].mode()
if not embarked_mode.empty:
    train['Embarked'] = train['Embarked'].fillna(embarked_mode[0])
```

```
In [13]: train = train.drop(['Cabin', 'Name', 'Ticket', 'PassengerId'], axis=1)
```

```
In [14]: train['Sex'] = train['Sex'].map({'male': 0, 'female': 1})

train = pd.get_dummies(train, columns=['Embarked'], drop_first=True)

print("Shape after preprocessing:", train.shape)
```

Shape after preprocessing: (891, 9)

Implementation

In [4]: `train.describe()`

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

In [5]: `train.isnull().sum()`

Out[5]:

PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

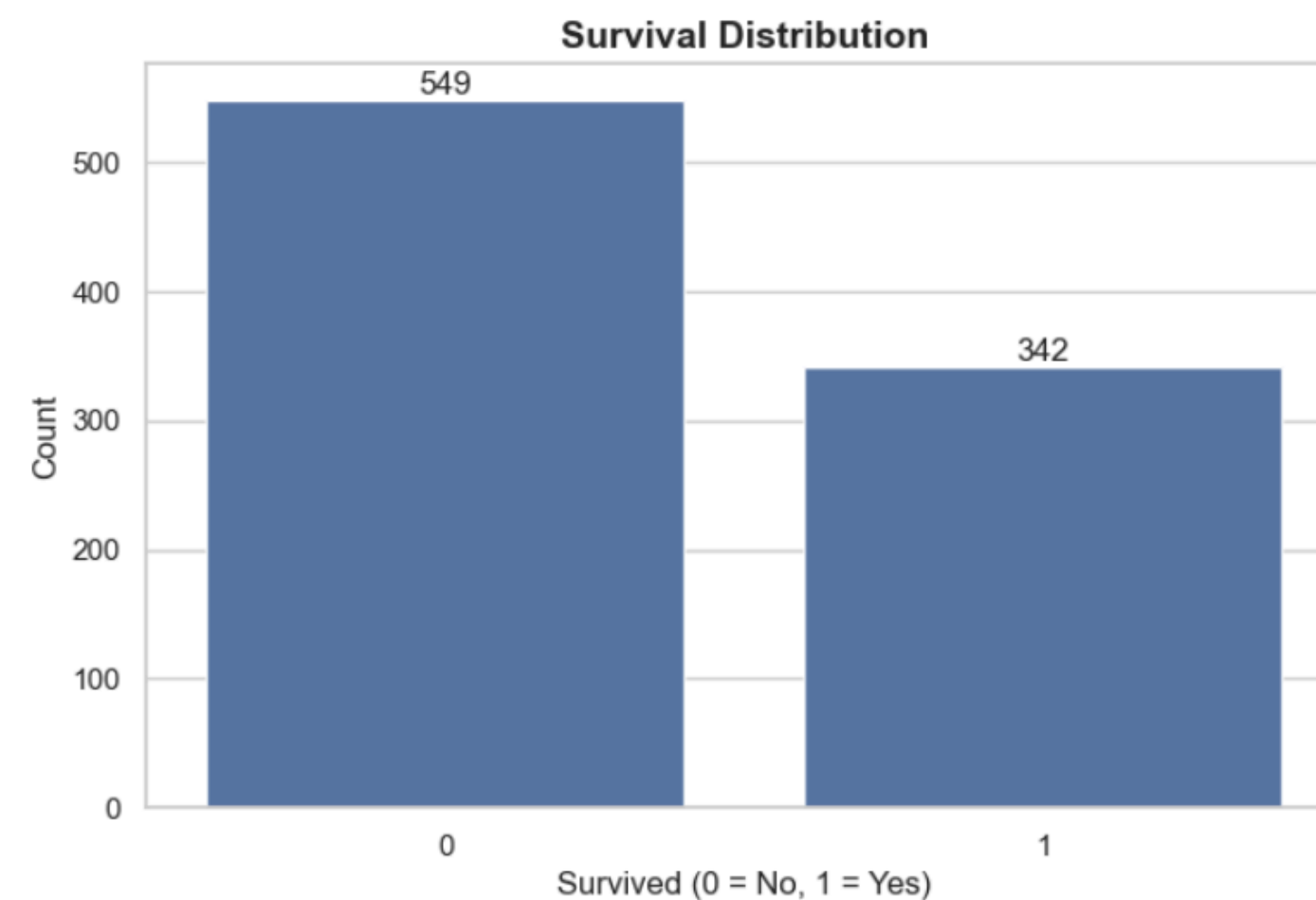
In [6]:

```
ax = sns.countplot(x='Survived', data=train)

plt.title("Survival Distribution", fontsize=14, weight='bold')
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Count")

for c in ax.containers:
    ax.bar_label(c)

plt.show()
```



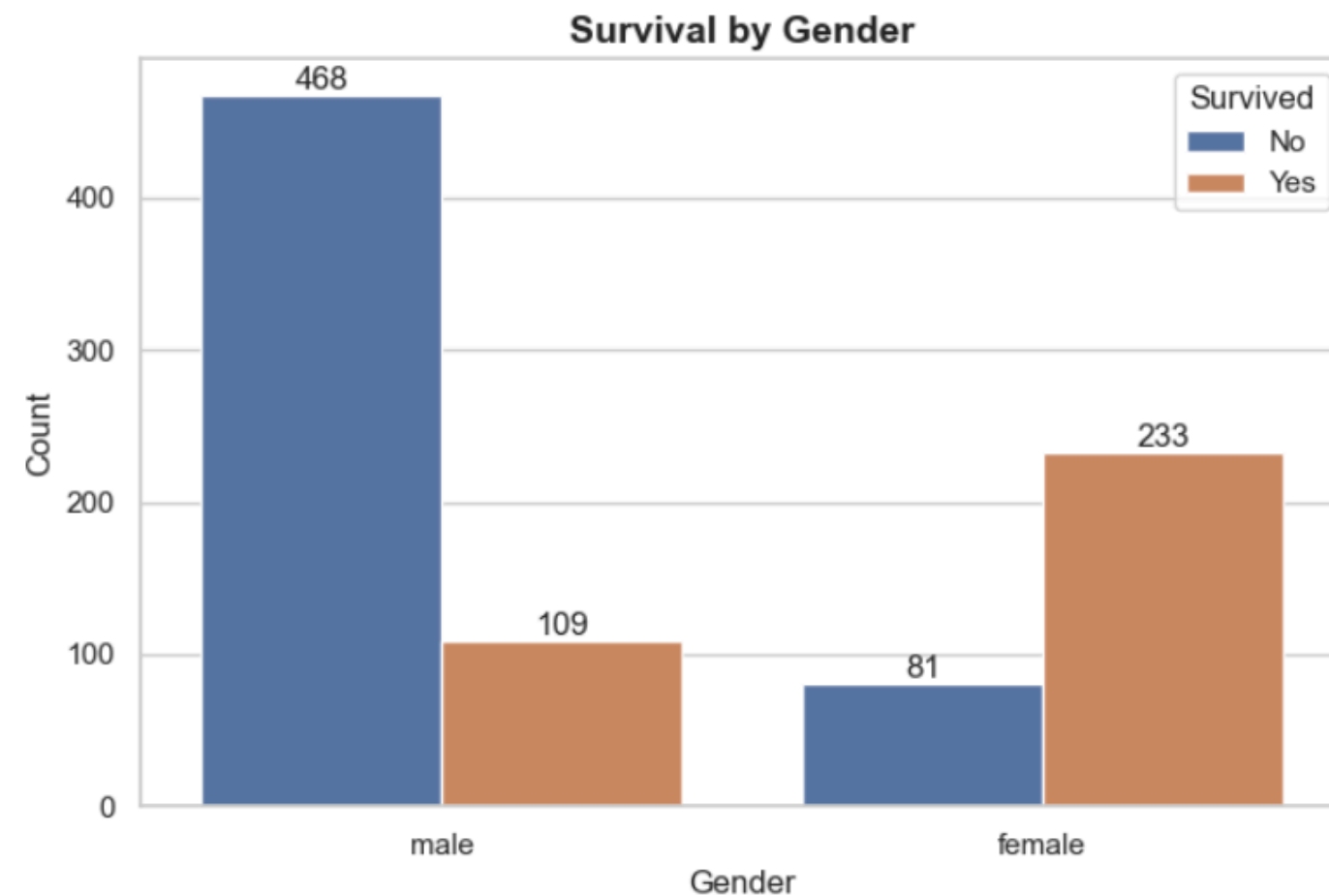
Implementation

```
In [7]: ax = sns.countplot(x='Sex', hue='Survived', data=train)

plt.title("Survival by Gender", fontsize=14, weight='bold')
plt.xlabel("Gender")
plt.ylabel("Count")
plt.legend(title="Survived", labels=["No", "Yes"])

for c in ax.containers:
    ax.bar_label(c)

plt.show()
```

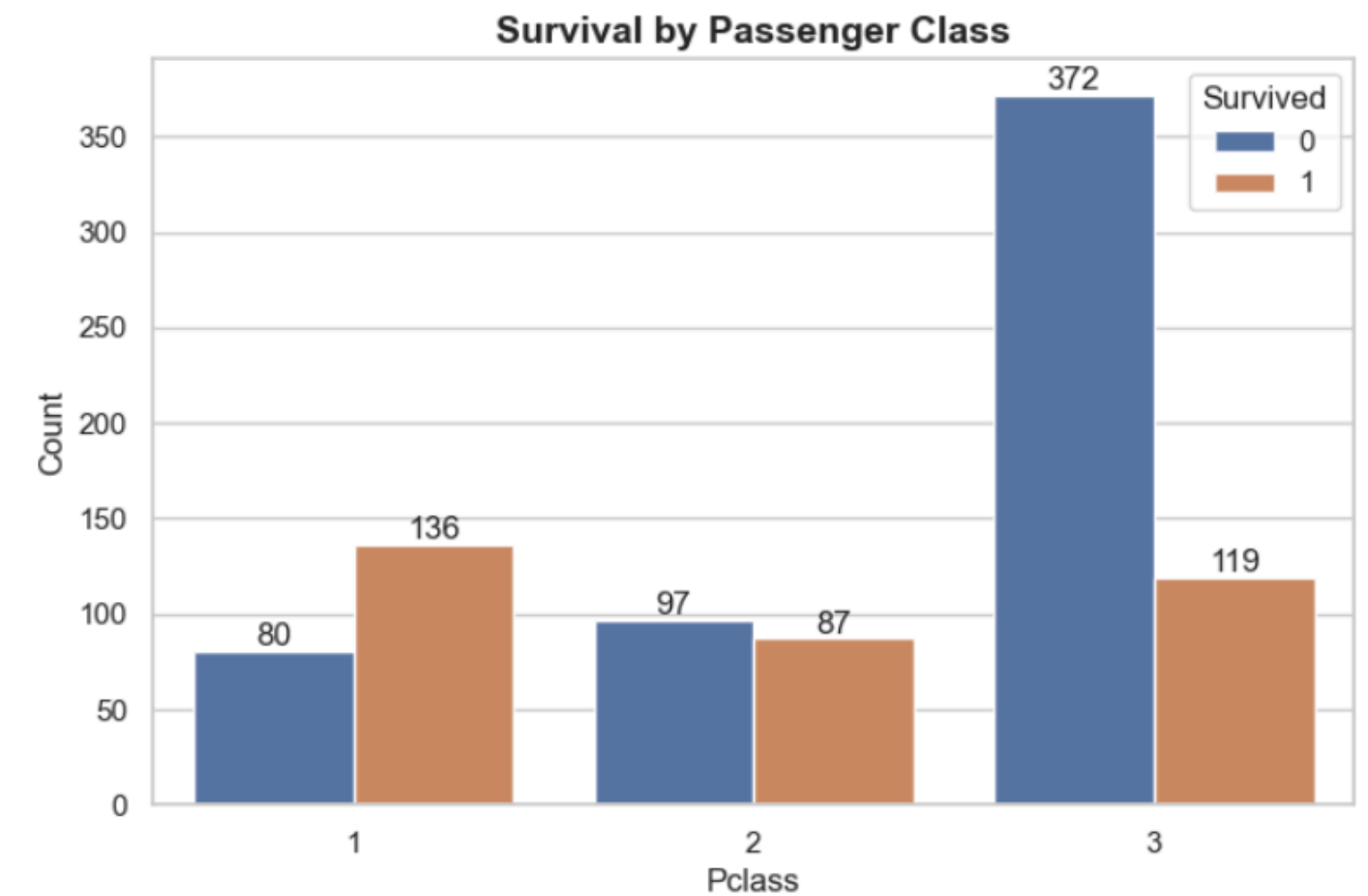


```
In [8]: ax = sns.countplot(x='Pclass', hue='Survived', data=train)

plt.title("Survival by Passenger Class", fontsize=14, weight='bold')
plt.xlabel("Pclass")
plt.ylabel("Count")

for c in ax.containers:
    ax.bar_label(c)

plt.show()
```



Implementation

GitHub Repository Link : https://github.com/Amitpatel-29/Titanic_Project

Deployed Link : <https://titanicprojectt.streamlit.app/>

Testing

In [20]:

```
print("✅ Model Training Completed")

print("\nIntercept:", model.intercept_)

coeff_df = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": model.coef_[0]
})

print("\nTop Features Influencing Survival:\n")
print(coeff_df.sort_values(by='Coefficient', ascending=False).head())

print("\nTraining Accuracy:", round(model.score(X_train, y_train), 2))

sample = X_train.iloc[0:1]
pred = model.predict(sample)

print("\nSample Prediction:", "Survived" if pred[0]==1 else "Not Survived")
```

✅ Model Training Completed

Intercept: [1.94369736]

Top Features Influencing Survival:

	Feature	Coefficient
1	Sex	2.591383
5	Fare	0.002579
2	Age	-0.030574
4	Parch	-0.108161
6	Embarked_Q	-0.112324

Training Accuracy: 0.8

Sample Prediction: Not Survived

Testing

```
In [18]: train_accuracy = model.score(X_train, y_train)

print("\nTraining Accuracy:", round(train_accuracy, 2))
```

Training Accuracy: 0.8

```
In [19]: sample = X_train.iloc[0:1]
prediction = model.predict(sample)

print("\nSample Input:\n", sample)
print("\nPrediction:", "Survived" if prediction[0]==1 else "Not Survived")
```

Sample Input:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked_Q	Embarked_S
331	1	0	45.5	0	0	28.5	False	True

Prediction: Not Survived

Testing

```
In [21]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

```
Accuracy: 0.8100558659217877
Precision: 0.7857142857142857
Recall: 0.7432432432432432
F1 Score: 0.7638888888888888
```

Testing

```
In [22]: from sklearn.metrics import roc_curve, roc_auc_score

y_prob = model.predict_proba(X_test)[: , 1]

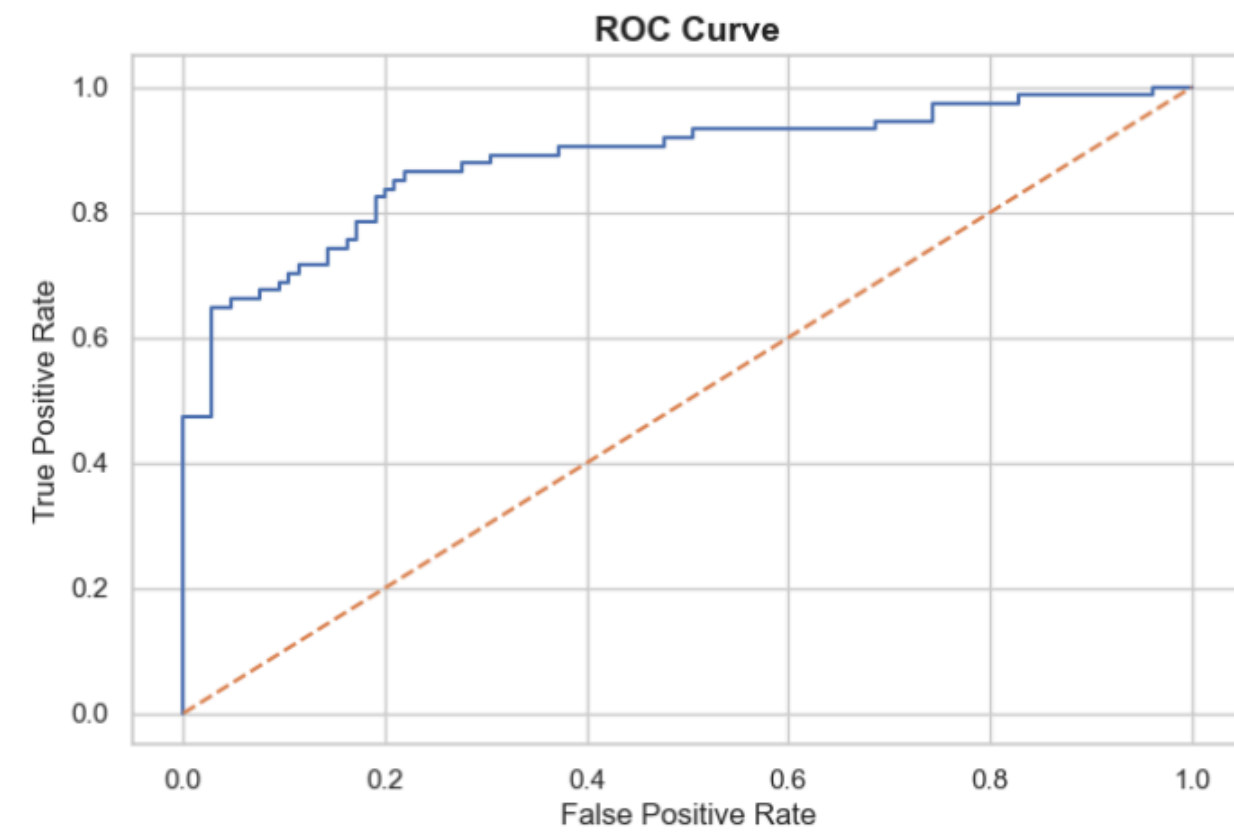
fpr, tpr, _ = roc_curve(y_test, y_prob)

plt.plot(fpr, tpr)
plt.plot([0,1], [0,1], linestyle='--')

plt.title("ROC Curve", fontsize=14, weight='bold')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.show()

print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))
```



ROC-AUC Score: 0.8824967824967825

Deployment

- Streamlit was used to develop an interactive web application.
- Users can enter passenger details through the interface.
- The application predicts passenger survival in real time.
- Prediction results are displayed instantly on the screen.

About Model

- Model: Logistic Regression
- Dataset: Titanic Dataset
- Target: Survival (0 = No, 1 = Yes)

Key Factors:

- Gender
- Passenger Class
- Fare

Share ☆ ↻ ⌵

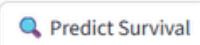
Titanic Survival Prediction App

This app predicts whether a passenger would **survive** or **not** based on input features using a **Logistic Regression** model.

👉 Enter the passenger details below and click **Predict**

Enter Passenger Details

Passenger Class	Siblings/Spouses
1	3
Sex	Parents/Children
male	1
Age	Fare
1	300.00
Embarked Port	
Q	

 Predict Survival

✓ Passenger is likely to SURVIVE

Probability: 0.59

Manage app

Deployment



Titanic Survival Prediction App

This app predicts whether a passenger would **survive** or **not** based on input features using a **Logistic Regression model**.

👉 Enter the passenger details below and click **Predict**



Enter Passenger Details

Passenger Class	Siblings/Spouses
1	3
Sex	Parents/Children
female	2
Age	Fare
37	241.94
Embarked Port	
Q	

🔍 Predict Survival

✅ Passenger is likely to SURVIVE

Probability: 0.83



Titanic Survival Prediction App

This app predicts whether a passenger would **survive** or **not** based on input features using a **Logistic Regression model**.

👉 Enter the passenger details below and click **Predict**



Enter Passenger Details

Passenger Class	Siblings/Spouses
2	3
Sex	Parents/Children
male	1
Age	Fare
1	300.00
Embarked Port	
Q	

🔍 Predict Survival

❌ Passenger is NOT likely to survive

Probability: 0.36



Titanic Survival Prediction App

This app predicts whether a passenger would **survive** or **not** based on input features using a **Logistic Regression model**.

👉 Enter the passenger details below and click **Predict**



Enter Passenger Details

Passenger Class	Siblings/Spouses
3	2
Sex	Parents/Children
male	2
Age	Fare
41	387.00
Embarked Port	
Q	

🔍 Predict Survival

❌ Passenger is NOT likely to survive

Probability: 0.09

Conclusion

End-to-End Implementation

Successfully developed a complete ML pipeline from data preprocessing to model deployment

Model Performance

Logistic Regression achieved ~81% accuracy with well-balanced precision and recall

Key Insights

Gender, passenger class, and age significantly influenced survival outcomes

Deployment

Deployed using Streamlit to provide real-time, user-friendly survival predictions

Future Scope

1 Expand Dataset

Include data from more ship disasters to improve model learning

3 Enhance Web App

Improve UI and add more interactive features

2 Diverse Data Collection

Use data from different ships to make predictions more general

4 Real-World Deployment

Deploy on cloud platforms for wider accessibility



References



Kaggle Titanic Dataset

Primary dataset for model training and analysis



Scikit-learn Documentation

Used for Logistic Regression and evaluation metrics



Pandas Library

Data cleaning, preprocessing, and transformation



NumPy Library

Primary dataset for model training and analysis



Matplotlib & Seaborn

Data visualization and exploratory analysis



Streamlit Documentation

Deployment of model as a web application

Thank You

THANK YOU FOR YOUR ATTENTION

